

MÔ PHỎNG CPU 8-CORE VỚI KIẾN TRÚC TẬP LỆNH RISC-V

Multiple Processing System sử dụng Verilog HDL

Thực hiện: Nhóm Học Viên

Báo Cáo Tiến Độ Dự Án Kiến Trúc Máy Tính

Ngày 18 tháng 6 năm 2026

Nội Dung Báo Cáo

1 Giới Thiệu Đề Tài

- Bối cảnh nghiên cứu
- Chủ đề của dự án
- Mục tiêu thiết kế
- Tại sao chọn Pipeline?

2 Kiến Trúc Tập Lệnh (ISA)

- Tập lệnh cơ bản RV32I
- Tập lệnh hiện thực trong dự án

3 Thiết Kế Hệ Thống (Design)

- Cấu trúc tổng thể
- Kiến trúc Pipeline và Logic Điều khiển

Nội Dung Báo Cáo

1 Giới Thiệu Đề Tài

- Bối cảnh nghiên cứu
- Chủ đề của dự án
- Mục tiêu thiết kế
- Tại sao chọn Pipeline?

2 Kiến Trúc Tập Lệnh (ISA)

3 Thiết Kế Hệ Thống (Design)

Bối Cảnh Nghiên Cứu & Lý Do Chọn Đề Tài

Giới hạn vật lý của CPU lõi đơn (Single-core)

Việc tăng hiệu năng bộ vi xử lý bằng cách tăng tần số xung nhịp đã đạt tới giới hạn vật lý do hiện tượng quá nhiệt và tiêu hao năng lượng cực lớn (**Power Wall, Thermal Wall**).

Chuyển dịch tất yếu sang Đa lõi (Multi-core)

Tích hợp nhiều lõi xử lý hoạt động độc lập trên cùng một chip để tối ưu hiệu năng tính toán song song.

Bài toán đặt ra

Giải quyết các vấn đề phức tạp về giao tiếp liên lõi, tranh chấp bus bộ nhớ dùng chung (memory contention) và đồng bộ giữa các luồng xử lý phần cứng.

Nội Dung Báo Cáo

1 Giới Thiệu Đề Tài

- Bối cảnh nghiên cứu
- Chủ đề của dự án
- Mục tiêu thiết kế
- Tại sao chọn Pipeline?

2 Kiến Trúc Tập Lệnh (ISA)

3 Thiết Kế Hệ Thống (Design)

Chủ Đề Của Dự Án

Dự án tập trung nghiên cứu, thiết kế phần cứng ở mức RTL sử dụng ngôn ngữ mô tả phần cứng **Verilog HDL** để mô phỏng bộ xử lý 8 lõi độc lập:

Kiến trúc nhân đơn

Mỗi lõi CPU dựa trên kiến trúc tập lệnh mở **RISC-V (RV32I subset)** áp dụng kỹ thuật **pipeline 5 tầng** nhằm tối ưu hóa thông lượng lệnh thực thi.

Kiến trúc đa nhân

Cả 8 nhân CPU kết nối và sử dụng chung một khối bộ nhớ dữ liệu (**Shared Data Memory**) thông qua một bộ điều khiển Bus và phân xử tập trung.

Nội Dung Báo Cáo

1 Giới Thiệu Đề Tài

- Bối cảnh nghiên cứu
- Chủ đề của dự án
- **Mục tiêu thiết kế**
- Tại sao chọn Pipeline?

2 Kiến Trúc Tập Lệnh (ISA)

3 Thiết Kế Hệ Thống (Design)

Mục Tiêu Thiết Kế

Dự án đề ra ba mục tiêu cốt lõi cần đạt được:

- **Thiết kế lõi đơn:** Thiết kế lõi xử lý RISC-V có pipeline 5 tầng hoàn chỉnh, xử lý xung đột lệnh (data/control hazards) một cách tối ưu bằng Forwarding, Stall và Flush.
- **Thiết kế trực bus đa lõi:** Xây dựng cơ chế phân xử công bằng (Round-Robin Arbiter), đảm bảo cả 8 nhân CPU đều có thể ghi/đọc bộ nhớ dùng chung mà không xảy ra xung đột hay deadlock.
- **Mô phỏng chức năng:** Sử dụng bộ công cụ nguồn mở **Icarus Verilog** để chạy mô phỏng hệ thống và **GTKWave** để phân tích giản đồ thời gian (waveform).

Nội Dung Báo Cáo

1 Giới Thiệu Đề Tài

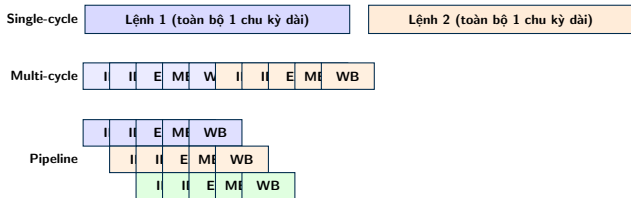
- Bối cảnh nghiên cứu
- Chủ đề của dự án
- Mục tiêu thiết kế
- Tại sao chọn Pipeline?

2 Kiến Trúc Tập Lệnh (ISA)

3 Thiết Kế Hệ Thống (Design)

Tại Sao Chọn Kiến Trúc Pipeline 5 Tầng?

So sánh 3 kiến trúc thực thi lệnh cơ bản:



Ưu điểm Pipeline

- Nhiều lệnh **chồng chéo** cùng tồn tại trong CPU ở các tầng khác nhau.
- Thông lượng lý tưởng: hoàn thành **1 lệnh mỗi chu kỳ** (so với 5 chu kỳ ở Multi-cycle).
- Tần số clock cao hơn Single-cycle vì mỗi tầng chỉ chứa một phần logic nhỏ.

Thách thức cần giải quyết

- **Data Hazard:** Lệnh sau cần kết quả của lệnh trước chưa hoàn tất.
- **Control Hazard:** Branch/Jump làm sai đường thực thi.
- **Structural Hazard:** Nhiều core tranh chấp cùng tài nguyên bộ nhớ.

Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
- 2 Kiến Trúc Tập Lệnh (ISA)
 - Tập lệnh cơ bản RV32I
 - Tập lệnh hiện thực trong dự án
- 3 Thiết Kế Hệ Thống (Design)

Tổng quan Tập lệnh RISC-V (RV32I)

RISC-V là gì?

Là một kiến trúc tập lệnh (ISA) mã nguồn mở dựa trên nguyên lý thiết kế máy tính tập lệnh rút gọn (RISC). Cho phép tự do thiết kế, tùy biến và tối ưu hóa phần cứng.

Giải nghĩa bộ lệnh cơ bản RV32I:

- **RV (RISC-V):** Tiêu chuẩn kiến trúc thế hệ thứ 5 từ UC Berkeley.
- **32 (32-bit):** Độ rộng thanh ghi và không gian địa chỉ 32-bit.
- **I (Integer):** Tập lệnh số nguyên cơ bản (gồm 40 lệnh). Đây là tập lệnh bắt buộc phải có cho mọi thiết kế chip RISC-V.

Mục tiêu lựa chọn

Đơn giản hóa thiết kế lõi đơn để tập trung tối ưu bộ điều khiển phân xử (Arbiter) và kết nối đa lõi (8-Core Interconnect).

Đặc trưng kỹ thuật RV32I

Các đặc trưng kỹ thuật:

- Không gian địa chỉ và độ dài lệnh cố định **32-bit**.
- Cơ chế **Load-Store**: Chỉ có lệnh Load/Store mới được truy cập Memory; các phép toán toán học chỉ thực thi trên Tập thanh ghi.
- **32 Thanh ghi đa năng** ($x0 \rightarrow x31$), trong đó $x0$ luôn cố định bằng số 0.
- 6 kiểu định dạng lệnh cơ bản: R, I, S, B, U, J.

Phân loại Tập thanh ghi (Register File)

- $x0$ (**zero**): Luôn bằng 0 (hằng số phần cứng).
- $x1$ (**ra**), $x2$ (**sp**): Địa chỉ trả về & Con trỏ Stack.
- $x3-x4$ (**gp-tp**): Con trỏ toàn cục và con trỏ tiểu trình.
- $x10-x17$ (**a0-a7**): Tham số truyền vào & Kết quả hàm.
- $x5-x7$, $x28-x31$ (**t0-t6**): Thanh ghi tạm thời.
- $x8-x9$, $x18-x27$ (**s0-s11**): Thanh ghi lưu giữ.

6 Kiểu định dạng lệnh cơ bản

	31	25	24	20	19	15	14	12	11	7	6	0				
R-type	funct7				rs2		rs1		funct3		rd		opcode			
I-type	imm[11:0]						rs1		funct3		rd		opcode			
S-type	imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode			
B-type	imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode			
U-type	imm[31:12]										rd		opcode			
J-type	imm[20 10:1 11 19:12]										rd		opcode			

Giải thích các trường

- **opcode**: Mã lệnh gốc (7 bit).
- **rd**: Thanh ghi đích.
- **funct3/7**: Mã mở rộng chức năng lệnh.
- **rs1/rs2**: Các thanh ghi nguồn.
- **imm**: Giá trị hằng số nạp trực tiếp.

Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
- 2 Kiến Trúc Tập Lệnh (ISA)
 - Tập lệnh cơ bản RV32I
 - Tập lệnh hiện thực trong dự án
- 3 Thiết Kế Hệ Thống (Design)

Tập lệnh hiện thực trong dự án

Tập Lệnh Số Nguyên RV32I Subset

40 lệnh cơ bản được hỗ trợ:

- **Tính toán (ALU):** ADD/SUB, SLL/SRL, AND/OR/XOR, SLT (bao gồm cả dạng hằng số Imm như ADDI, ANDI,...).
- **Bộ nhớ:** LW (Load Word), SW (Store Word).
- **Điều khiển luồng:** BEQ, BNE, BLT, BGE, JAL, JALR.
- **Khác:** LUI, AUIPC.

Đồng Bộ Đa Nhân (A-Extension)

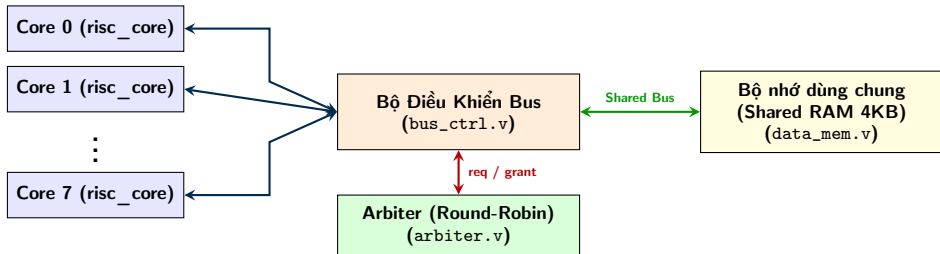
Thiết kế riêng cho hệ thống 8-Core:

- **Độc quyền truy cập:** LR.W (Load-Reserved) và SC.W (Store-Conditional) để xây dựng cấu trúc đồng bộ (Mutex/Semaphore).
- **Phép toán nguyên tử (AMO):** AMOSWAP, AMOADD, AMOAND, AMOOR, AMOXOR, AMOMIN/MAX trực tiếp trên bộ nhớ dùng chung.

Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
- 2 Kiến Trúc Tập Lệnh (ISA)
- 3 **Thiết Kế Hệ Thống (Design)**
 - **Cấu trúc tổng thể**
 - Kiến trúc Pipeline và Logic Điều khiển

Cấu Trúc Tổng Thể Hệ Thống 8-Core



Hệ thống kết nối đa nhân: **8 lõi CPU** hoạt động song song độc lập, thực hiện phân xử quyền truy cập bộ nhớ thông qua bộ phân xử **Arbiter** và bộ định tuyến bus dữ liệu **Bus Controller**.

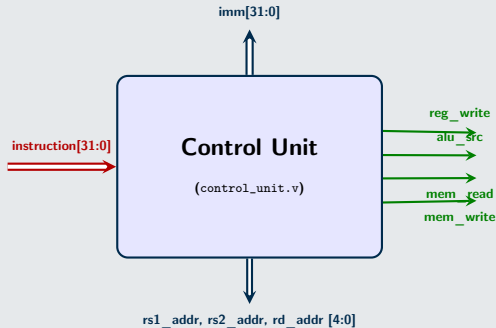
Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
- 2 Kiến Trúc Tập Lệnh (ISA)
- 3 Thiết Kế Hệ Thống (Design)
 - Cấu trúc tổng thể
 - Kiến trúc Pipeline và Logic Điều khiển

Control Unit

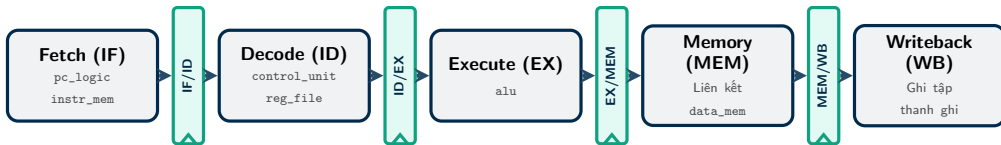
- **Nhiệm vụ:** Sử dụng Combinational Logic để giải mã các trường Opcode, Funct3, Funct7 của Instruction và lập tức sinh các tín hiệu điều khiển cho hệ thống Datapath.

Sơ đồ Input Output Control Unit



Phân bố phần cứng trên 5 tầng Pipeline

Hệ thống **Pipeline Registers** đóng vai trò là các vách ngăn đồng bộ, cô lập dữ liệu giữa các giai đoạn xử lý:



Nguyên lý điều khiển luồng trên các Thanh ghi đệm Pipeline

- **Clock Synchronization:** Dữ liệu/lệnh từ tầng trước chỉ được phép dịch chuyển sang tầng sau chính xác tại sườn lên của xung Clock (posedge clk).
- **Pipeline register:** Giữ cố định trạng thái xử lý độc lập của 5 lệnh khác nhau cùng lúc, ngăn chặn hiện tượng lẫn át hay trộn lẫn dữ liệu giữa các tầng Datapath.
- **Pipeline Interlock:** Là nơi trực tiếp tiếp nhận các lệnh (hold để tạo Stall, hoặc clear để tạo Flush/Bubble) từ bộ điều khiển trung tâm.

PIPELINE REGISTERS: Fetch / Decode (IF/ID Register)

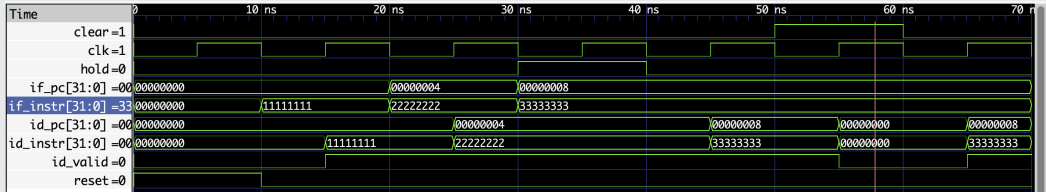
- **Vai trò:** Chốt giữ Instruction 32-bit vừa đọc ra từ Instruction Memory và địa chỉ PC tương ứng để chuyển giao sang giai đoạn Decode.
- **Cơ chế Hazard:** Nhận tín hiệu hold khi kích hoạt Load-Use Stall hoặc Memory Stall để đồng bộ trạng thái; nhận tín hiệu clear để thực hiện Flush lệnh rác khi rẽ nhánh Taken sai.

Sơ đồ Input Output- if_id.v



Pipeline Registers Unit Testing - IF/ID

1. Kết quả Simulation if_id

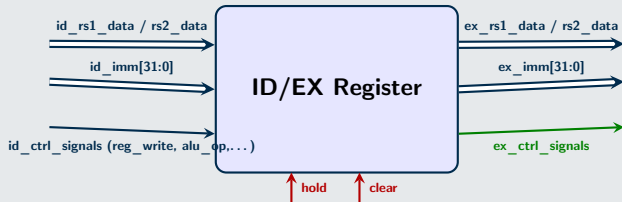


Phân tích Waveform: Xác thực phản hồi chính xác khi tín hiệu hold dựng lên, giá trị PC và Instruction được khóa cứng, không bị ảnh hưởng bởi sườn Clock.

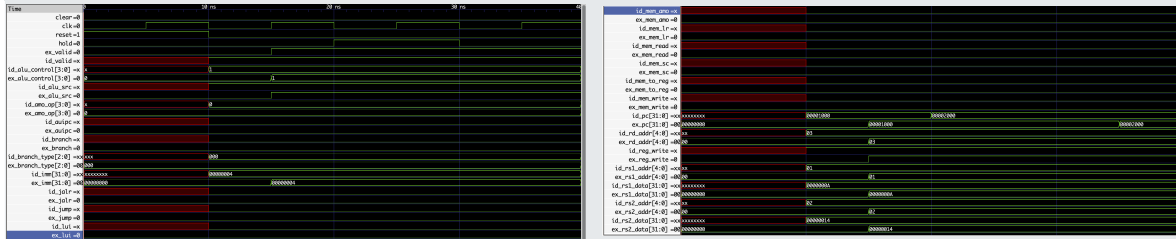
PIPELINE REGISTERS Decode / Execute (ID/EX Register)

- **Vai trò:** Tiếp nhận và chuyển tiếp toàn bộ các tín hiệu điều khiển từ Control Unit, giá trị các toán hạng đọc từ Register File (`rs1_data`, `rs2_data`) và số Immediate đã mở rộng sang giai đoạn Execute cho ALU.
- **Cơ chế Hazard:** Nhận tín hiệu `clear = 1` để chủ động chèn một Bubble (NOP) vào tầng Execute khi xảy ra xung đột Load-Use.

Sơ đồ Input Output - id_ex.v



2. Kết quả Simulation id_ex

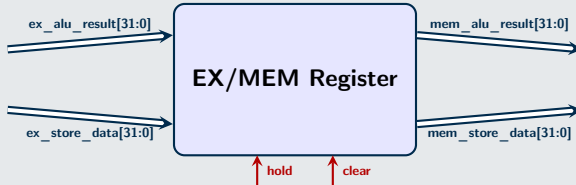


Phân tích Waveform: Tín hiệu clear (Flush) kích hoạt thành công, xóa sạch các cờ điều khiển về trạng thái 0 (NOP Bubble) ngay chu kỳ kế tiếp.

PIPELINE REGISTERS Execute / Memory (EX/MEM Register)

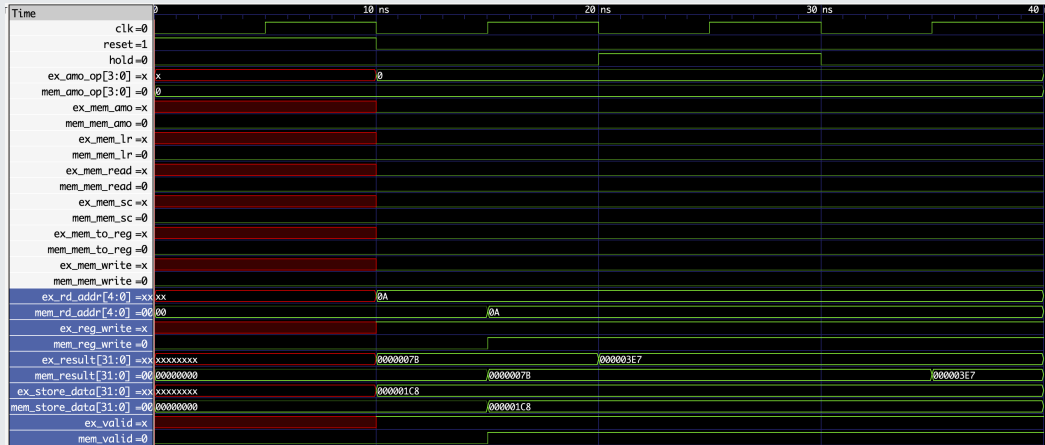
- **Vai trò:** Chốt giữ kết quả tính toán từ đầu ra của ALU, dữ liệu cần ghi vào Data Memory (rs2_data) của lệnh Store, và địa chỉ thanh ghi đích rd.
- **Cơ chế Hazard:** Nhận tín hiệu hold từ mạch Memory Stall (mem_wait) nhằm giữ nguyên thông tin khi Shared Bus đang bận phân xử quyền truy cập bộ nhớ đa nhân.

Sơ đồ Input Output - ex_mem.v



Pipeline Registers Unit Testing - EX/MEM

3. Kết quả Simulation ex_mem

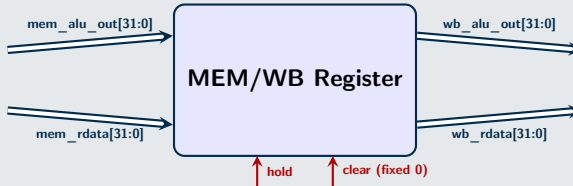


Phân tích Waveform: Các toán hạng từ ALU và dữ liệu Store được chốt giữ và chuyển giao đồng bộ

PIPELINE REGISTERS Memory / Writeback (MEM/WB Register)

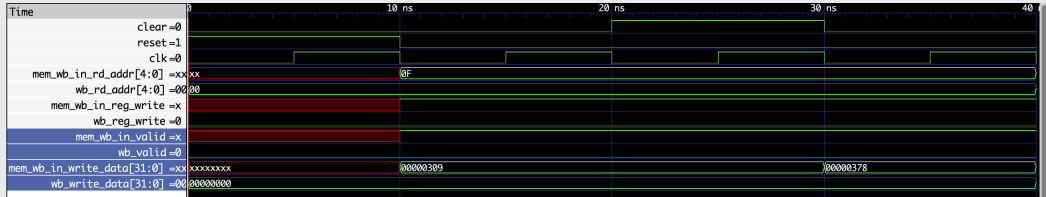
- **Vai trò:** Chốt giữ dữ liệu đọc lên từ Data Memory (rdata) hoặc kết quả trực tiếp từ ALU để sẵn sàng đưa về cổng ghi của Register File.
- **Cơ chế Hazard (Đã vá lỗi):** Loại bỏ hoàn toàn chân clear độc hại, chuyển sang sử dụng chân hold đồng bộ. Khi gặp Memory Stall, dữ liệu tầng này đứng im để bảo toàn dữ liệu, tránh lỗi mất thông tin trên Pipeline.

Sơ đồ Input Output - mem_wb.v



Pipeline Registers Unit Testing - MEM WB

4. Kết quả Simulation mem_wb

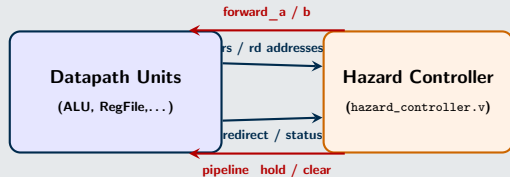


Phân tích Waveform: Cơ chế vá lỗi thực tế; sử dụng chân hold thay vì clear. Khi xảy ra Memory Stall (mem_wait), dữ liệu tầng này đứng im để bảo toàn tính toàn vẹn.

Nguyên tắc đóng gói:

- Giữ nguyên 100% giao tiếp Ports ra hệ thống Shared Bus ngoại vi và mô hình 8-Core.
- Thực hiện instantiate 6 module con độc lập bên trong ruột và kết nối đồng bộ bằng mạng lưới wire.

Sơ đồ kết nối khối điều khiển trung tâm trong Core



Hazard Detection & Forwarding

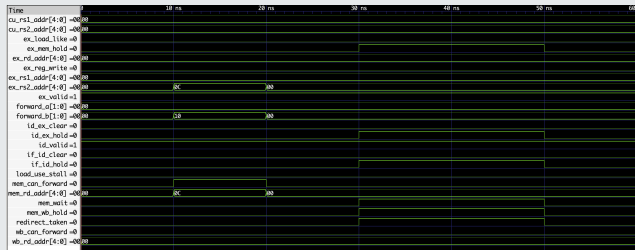
- **Tư duy tích hợp:** Tập trung hóa (Centralized Control) toàn bộ logic xử lý Load-Use Stall, đi tắt dữ liệu (Forwarding) và quản lý trạng thái bận hệ thống (mem_wait) vào một bộ điều khiển duy nhất nhằm tối ưu tài nguyên phần cứng.

Sơ đồ khối điều khiển trung tâm hệ thống - Hazard Controller



Hazard Controller Unit Testing

Kết quả Simulation trên GTKWave



- Thử nghiệm thành công trên hai kịch bản tối hạn (Critical Scenarios).

Phân tích định thời Waveform:

1. Data Forwarding Interval

Tại khoảng thời gian 10ns - 20ns:

- Cặp cổng giám sát phát hiện `ex_rs2_addr` trùng với `mem_rd_addr` (0x0C).
- `forward_b` ngay lập tức phản hồi mã 2'b10, thực hiện đi tắt dữ liệu từ tầng EX/MEM.

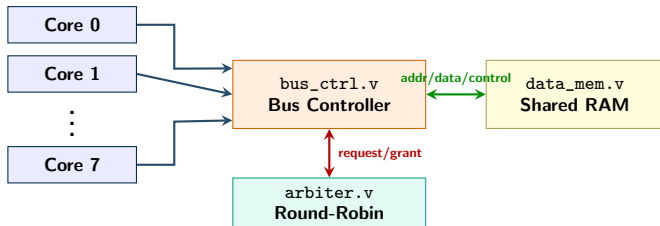
2. Memory Stall & Control Flush

Tại khoảng thời gian 30ns - 50ns:

- Khi `mem_wait` và `redirect_taken` đồng loạt dựng lên mức 1.
- Hệ thống dựng toàn bộ các cờ hold để đóng băng Pipeline.
- Cờ clear bị chặn về 0, bảo vệ an toàn lệnh chờ bộ nhớ khỏi bị xóa nhầm.

Vai Trò Của Interconnect Trong Hệ Thống 8 Core

Khi 8 core cùng cần đọc/ghi data_mem, interconnect đóng vai trò cầu nối để chọn đúng core, đúng địa chỉ và trả phản hồi về đúng nơi.



Nhóm tín hiệu chính

- Core yêu cầu: `core_mem_req`, `core_addr`, `core_wdata`.
- Kiểu giao dịch: `core_we`, `core_re`, `core_lr`, `core_sc`, `core_amo`.
- Phản hồi: `core_ready`, `core_rdata`, `core_sc_result`.

Ý chính

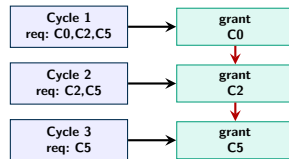
Interconnect không xử lý hazard pipeline; nó cấp quyền truy cập RAM và tạo tín hiệu sẵn sàng để core tiếp tục chạy.

Arbiter Round-Robin: Chia Quyền Truy Cập Bus

arbiter.v nhận vector request[7:0] và chọn một core được cấp bus trong mỗi lượt cấp quyền.

Cơ chế cấp quyền

- grant[7:0] là one-hot, chỉ core thắng arbitration được bật bit.
- grant_valid báo grant hợp lệ trong một chu kỳ.
- grant_id cho bus_ctrl biết core nào đang được phục vụ.
- priority_ptr dịch sang core kế tiếp sau mỗi grant.
- last_granted tránh cấp lại ngay cho cùng request cũ.

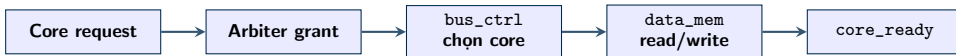


Mục tiêu

Chia sẻ RAM công bằng hơn giữa các core, giảm nguy cơ một core chiếm bus liên tục.

Bus Controller: Định Tuyển Giao Dịch Read/Write

Sau khi arbiter cấp quyền, `bus_ctrl.v` chọn tín hiệu của core thắng và điều khiển giao tiếp với `data_mem`.



Read

- Bật `dm_re` và đưa `dm_addr` sang RAM.
- Nhận `dm_rdata` từ memory interface.
- Ghi dữ liệu vào `core_rdata[grant_id]` và bật `core_ready`.

Write

- Bật `dm_we`, đưa `dm_addr` và `dm_wdata`.
- `data_mem` ghi dữ liệu theo cạnh clock.
- `bus_ctrl` báo `core_ready` để core kết thúc giao dịch.

Trong thiết kế hiện tại, read ở memory interface là combinational/async; write là synchronous trong `data_mem`.

Atomic LR/SC Và AMO Trong Interconnect

Interconnect có primitive phần cứng cho đồng bộ đa lõi thông qua reservation và phép toán nguyên tử trên RAM dùng chung.

LR/SC Reservation

- LR.W: đọc memory và đặt `reservation_valid[core]` tại `reservation_addr[core]`.
- SC.W thành công nếu reservation còn đúng địa chỉ; khi đó RAM được ghi và kết quả trả về là thành công.
- Nếu core khác ghi cùng địa chỉ, reservation bị invalidate; SC.W sau đó thất bại.

AMO

- AMO đọc giá trị cũ từ RAM.
- `bus_ctrl` tính giá trị mới theo `amo_op`: add, swap, and, or, xor, min/max.
- RAM nhận giá trị mới; core nhận lại old value qua `core_rdata`.

Đây là nền tảng phần cứng để xây dựng cơ chế đồng bộ; slide này không trình bày như một mutex/semaphore phần mềm hoàn chỉnh.

Testbench Và Kết Quả Kiểm Chứng Interconnect

Kiểm chứng tập trung ở tb/interconnect_tb.v, tách riêng khỏi pipeline core để kiểm tra trực tiếp arbiter, bus_ctrl và data_mem.

Test	Mục tiêu kiểm chứng	Kết quả
RR Write/Read	4 core ghi/đọc các địa chỉ khác nhau qua bus dùng chung.	PASS
LR/SC success	Core thực hiện LR.W rồi SC.W khi không có core khác ghi chen vào.	PASS
LR/SC fail	Core khác ghi cùng địa chỉ làm reservation bị xóa trước SC.W.	PASS
AMOADD	Hai core cộng nguyên tử, RAM cuối cùng nhận đúng tổng giá trị.	PASS

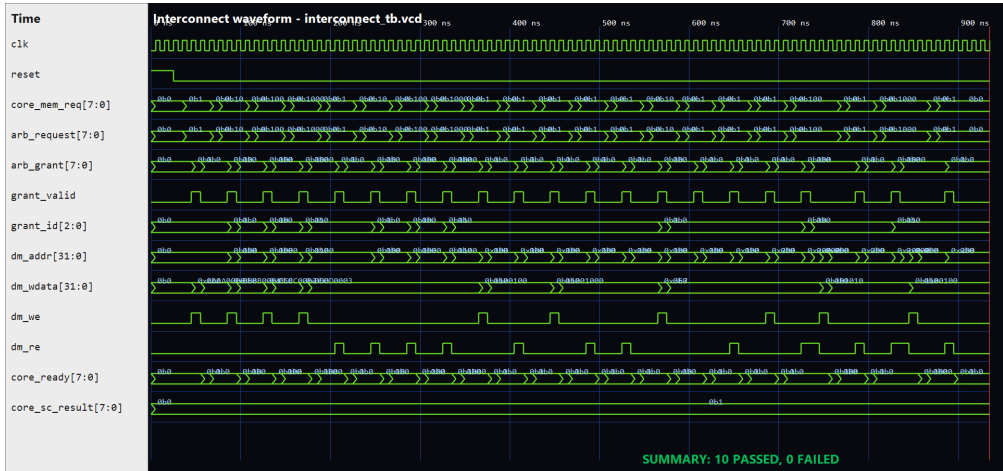
Kết quả mô phỏng

SUMMARY: 10 PASSED, 0 FAILED

Tín hiệu kiểm chứng chính

core_mem_req, arb_request, arb_grant, grant_id, dm_addr, dm_we, dm_re, core_ready, core_sc_result.

Waveform Kiểm Chứng Interconnect



Minh chứng luồng core_mem_req -> arb_grant -> dm_we/dm_re -> core_ready và kết quả 10 PASSED, 0 FAILED.

Phân Tích Kết Quả Kiểm Chứng Interconnect

Dựa trên kết quả mô phỏng từ `interconnect_tb.vcd`, khối `interconnect` vượt qua 10/10 kiểm tra trong testbench riêng, chứng minh đúng các chức năng chính trong phạm vi đề án:

1. Chia sẻ Bus (Round-Robin)

Các core gửi request ghi/đọc qua bus dùng chung. Tín hiệu `arb_grant` cấp quyền theo lượt, cho thấy arbiter chọn đúng core đang yêu cầu và không để một request hợp lệ bị bỏ qua trong test.

2. Định tuyến dữ liệu (Bus Control)

Tín hiệu `dm_addr`, `dm_we`, `dm_wdata` được định tuyến từ core được grant tới Shared RAM. Khi giao dịch hoàn tất, `core_ready` bật để báo core nhận được phản hồi.

3. Đồng bộ hóa đa lỗi (LR/SC & AMO)

- **Thành công nguyên tử:** Lệnh SC ghi thành công khi không có can thiệp.
- **Phát hiện xung đột:** Khi C0 đang giữ reservation (LR), C1 ghi đè vào địa chỉ đó. Bộ điều khiển lập tức xóa reservation, khiến lệnh SC tiếp theo của C0 **thất bại an toàn**, bảo vệ tính toàn vẹn dữ liệu dùng chung.
- **AMOADD:** Hai phép cộng nguyên tử cập nhật RAM đúng kết quả cuối $42 + 58 = 100$.

Kết Luận

Trong phạm vi testbench, `interconnect` đã chứng minh được ba nhiệm vụ chính: phân xử bus, định tuyến dữ liệu bộ nhớ và hỗ trợ primitive đồng bộ LR/SC/AMOADD.